გაკვეთილი 28 mod 3 (3.5)

გაკვეთილი 3 — DOM ელემენტების სტილის მართვა JavaScript-ით

1. თეორიული მასალა

1.1. როგორ ვცვლით HTML ელემენტების სტილს JavaScript-ით?

წინა გაკვეთილებზე ვისწავლეთ, როგორ მოვძებნოთ HTML ელემენტები და შევცვალოთ მათი ტექსტი ან შიგთავსი.

მაგალითად:

```
let title = document.querySelector("#title");

title.textContent = "Hello JavaScript!";
```

ახლა ვისწავლით, როგორ შევცვალოთ ამავე ელემენტის ვიზუალური მხარე.

JavaScript-ის საშუალებით შეგვიძლია შევცვალოთ:

- ტექსტის ფერი;
- ფონის ფერი;
- ტექსტის ზომა;
- ელემენტის სიგანე და სიმაღლე;
- საზღვრები და მომრგვალება;
- ელემენტის მდებარეობა და დაშორებები;
- CSS კლასები.

ამისთვის არსებობს ორი ძირითადი მიდგომა:

1. `element.style` — კონკრეტული CSS თვისებების შეცვლა.
2. `element.classList` — CSS კლასების დამატება, წაშლა და მართვა.

2. `element.style` — CSS სტილების შეცვლა

`style` არის HTML ელემენტის თვისება, რომლის საშუალებითაც JavaScript-იდან შეგვიძლია ელემენტს მივანიჭოთ CSS სტილები.

ზოგადი სინტაქსი:

```
element.style.property = "value";
```

აქ:

- `element` — მოძიებული HTML ელემენტია.
- `style` — საშუალებას გვაძლევს ვიმუშაოთ მის CSS სტილებთან.
- `property` — CSS თვისებაა.
- `value` — მნიშვნელობა, რომელიც უნდა მივანიჭოთ.

2.1. ტექსტის ფერის შეცვლა — `color`

HTML:

```
<h1 id="title">Hello World</h1>
```

JavaScript:

```
let title = document.querySelector("#title");

title.style.color = "blue";
```

შედეგი:



შეგვიძლია გამოვიყენოთ სხვადასხვა ფერის მნიშვნელობები:

```
title.style.color = "red";

title.style.color = "#00ff00";

title.style.color = "rgb(255, 100, 0)";
```

სამივე მაგალითი ფერის განსაზღვრის განსხვავებული ფორმაა.

2.2. ფონის შეცვლა — `backgroundColor`

HTML:

```
<div id="box">Hello Students!</div>
```

JavaScript:

```
let box = document.querySelector("#box");

box.style.backgroundColor = "lightblue";
```

შედეგი:



მნიშვნელოვანი: CSS-ში ხშირად ვიყენებთ თვისებას `background-color`, ხოლო JavaScript-ის `style` ობიექტში იგივე თვისება იწერება `backgroundColor`.

ეს იმიტომ ხდება, რომ JavaScript-ში გამოიყენება `camelCase` ჩაწერის სტილი.

CSS	JavaScript
`background-color`	`backgroundColor`
`font-size`	`fontSize`
`border-radius`	`borderRadius`
`text-align`	`textAlign`
`margin-top`	`marginTop`
`font-weight`	`fontWeight`

2.3. ტექსტის ზომის შეცვლა — `fontSize`

HTML:

```
<p id="message">Learning JavaScript</p>
```

JavaScript:

```
let message = document.querySelector("#message");
```

```
message.style.fontSize = "30px";
```

განმარტება:

- `fontSize` განსაზღვრავს ტექსტის ზომას.
- `"30px"` არის CSS მნიშვნელობა.
- მნიშვნელობა უნდა გადავცეთ სტრიქონის სახით, შესაბამისი საზომი ერთეულით.

შეგვიძლია გამოვიყენოთ:

```
message.style.fontSize = "20px";

message.style.fontSize = "2rem";

message.style.fontSize = "150%";
```

2.4. რამდენიმე CSS თვისების ერთდროულად შეცვლა

ერთ HTML ელემენტს შეგვიძლია რამდენიმე სტილი მივანიჭოთ.

HTML:

```
<div id="card">
    <h2>My Card</h2>
    <p>This is my first card.</p>
</div>
```

JavaScript:

```
let card = document.querySelector("#card");

card.style.backgroundColor = "lightblue";
card.style.color = "darkblue";
card.style.padding = "20px";
card.style.borderRadius = "12px";
card.style.width = "250px";
card.style.border = "2px solid blue";
```

შედეგი:



ამ მაგალითში `style` თვისების გამოყენებით შევცვალეთ ბარათის ფონი, ტექსტის ფერი, შიდა დაშორება, სიგანე, საზღვარი და კუთხეების მომრგვალება.

3. რამდენიმე ელემენტის სტილის შეცვლა

ზოგჯერ გვჭირდება, რომ ერთდროულად რამდენიმე HTML ელემენტს შევუცვალოთ დიზაინი.

ამისთვის შეგვიძლია გამოვიყენოთ `querySelectorAll()` და `forEach()`.

HTML:

```
<p class="text">First paragraph</p>
<p class="text">Second paragraph</p>
<p class="text">Third paragraph</p>
```

JavaScript:

```
let paragraphs = document.querySelectorAll(".text");

paragraphs.forEach((paragraph) => {
    paragraph.style.color = "green";
    paragraph.style.fontSize = "20px";
});
```

შედეგად სამივე პარაგრაფს ექნება მწვანე ტექსტი და 20 პიქსელის ზომა.

მაგალითი — განსხვავებული სტილები ინდექსის მიხედვით

```
let paragraphs = document.querySelectorAll(".text");

paragraphs.forEach((paragraph, index) => {

    if (index === 0) {
        paragraph.style.color = "red";
    } else if (index === 1) {
        paragraph.style.color = "blue";
    } else {
```

```
        paragraph.style.color = "green";
    }

});
```

ამ შემთხვევაში თითოეულ პარაგრაფს განსხვავებული ფერი მიენიჭება.

ეს განსაკუთრებით გამოსადეგია სიებთან, ბარათებთან და სხვა განმეორებად ელემენტებთან მუშაობისას.

4. `classList` — CSS კლასების მართვა

`classList` გამოიყენება HTML ელემენტის CSS კლასების სამართავად.

`style` თვისებისგან განსხვავებით, `classList` საშუალებას გვაძლევს JavaScript-იდან გამოვიყენოთ წინასწარ CSS-ში განსაზღვრული სტილები.

მაგალითად, გვაქვს ასეთი CSS:

```
.highlight {
    background-color: yellow;
    color: black;
    padding: 15px;
    border-radius: 10px;
}
```

JavaScript-ით შეგვიძლია ელემენტს ეს კლასი მივანიჭოთ და ყველა სტილი ერთდროულად გამოვიყენოთ.

`classList`-ის ძირითადი მეთოდებია:

მეთოდი	დანიშნულება
`classList.add()`	ამატებს CSS კლასს
`classList.remove()`	შლის CSS კლასს
`classList.toggle()`	ამატებს ან შლის კლასს
`classList.contains()`	ამოწმებს, აქვს თუ არა ელემენტს კლასი

4.1. `classList.add()` — კლასის დამატება

HTML:

```
<h1 id="title">Hello Students!</h1>
```

CSS:

```
.highlight {
    color: white;
    background-color: blue;
    padding: 20px;
    border-radius: 10px;
}
```

JavaScript:

```
let title = document.querySelector("#title");

title.classList.add("highlight");
```

შედეგი:



განმარტება:

1. `querySelector()` პოულობს ელემენტს.
2. `classList` გვაძლევს მის კლასებთან მუშაობის საშუალებას.
3. `add("highlight")` ამატებს `highlight` კლასს.
4. ბრაუზერი ავტომატურად იყენებს ამ კლასისთვის CSS-ში განსაზღვრულ სტილებს.

შეგვიძლია ერთდროულად რამდენიმე კლასის დამატებაც:

```
title.classList.add("highlight", "large-text");
```

4.2. `classList.remove()` — კლასის წაშლა

`remove()` გამოიყენება HTML ელემენტიდან კონკრეტული CSS კლასის მოსაშორებლად.

HTML:

```
<p id="message" class="highlight">
    Important Message
```

```
</p>
```

JavaScript:

```
let message = document.querySelector("#message");

message.classList.remove("highlight");
```

ამ კოდის შესრულების შემდეგ ელემენტს `highlight` კლასი აღარ ექნება.

შესაბამისად, ამ კლასით მინიჭებული სტილებიც აღარ იმოქმედებს, თუ ისინი სხვა წესებით არ არის განსაზღვრული.

4.3. `classList.toggle()` — კლასის გადართვა

`toggle()` განსაკუთრებით გამოსადეგი მეთოდია.

იგი ამოწმებს, აქვს თუ არა ელემენტს კონკრეტული კლასი:

- თუ კლასი არ აქვს — დაამატებს.
- თუ კლასი აქვს — წაშლის.

HTML:

```
<div id="box">My Box</div>
```

CSS:

```
.active {
    background-color: green;
    color: white;
    padding: 20px;
}
```

JavaScript:

```
let box = document.querySelector("#box");

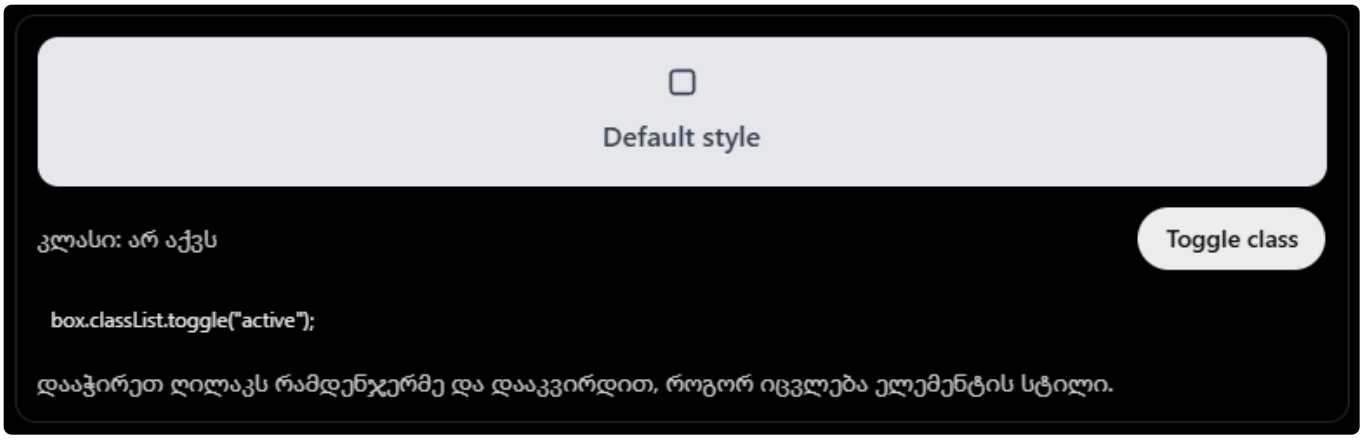
box.classList.toggle("active");
```

თუ `box` ელემენტს თავდაპირველად არ აქვს `active` კლასი, პირველი გამოძახება მას დაამატებს.

```
box.classList.toggle("active");
```

მეორე გამოძახება კი იმავე კლასს წაშლის.

ინტერაქტიული მაგალითი:



4.4. `classList.contains()` — კლასის არსებობის შემოწმება

`contains()` ამოწმებს, აქვს თუ არა ელემენტს მითითებული CSS კლასი.

მეთოდი აბრუნებს `true` ან `false` მნიშვნელობას.

HTML:

```
<p id="text" class="important">
    Important information
</p>
```

JavaScript:

```
let text = document.querySelector("#text");

console.log(text.classList.contains("important"));
```

შედეგი:

```
true
```

ხოლო:

```
console.log(text.classList.contains("active"));
```

შედეგი:

```
false
```

შეგვიძლია გამოვიყენოთ პირობით ოპერატორთან ერთად:

```
if (text.classList.contains("important")) {
    console.log("This text is important");
} else {
    console.log("This text is not important");
}
```

5. `style` და `classList` — განსხვავება

ორივე გამოიყენება HTML ელემენტების ვიზუალური მხარის სამართავად, თუმცა განსხვავებული მიდგომებით.

`style`	`classList`
ცვლის კონკრეტულ CSS თვისებას	მართავს CSS კლასებს
სტილი იწერება JavaScript-ში	სტილი წინასწარ იწერება CSS-ში
მოსახერხებელია ინდივიდუალური ცვლილებისთვის	მოსახერხებელია რამდენიმე სტილის ერთდროულად გამოყენებისთვის
`element.style.color = "red"`	`element.classList.add("red-text")`

მაგალითი — `style`-ით:

```
let card = document.querySelector(".card");

card.style.backgroundColor = "blue";
card.style.color = "white";
card.style.padding = "20px";
card.style.borderRadius = "10px";
```

იგივე მაგალითი — `classList`-ით:

CSS:

```
.active-card {
    background-color: blue;
    color: white;
```

```
    padding: 20px;
    border-radius: 10px;
}
```

JavaScript:

```
let card = document.querySelector(".card");

card.classList.add("active-card");
```

ორივე შემთხვევაში ვიზუალური შედეგი ერთნაირი იქნება.

რომელი მიდგომა ჯობს?

როდესაც მხოლოდ ერთი თვისების შეცვლა გვინდა, `style` მარტივი გამოსაყენებელია. როდესაც რამდენიმე სტილის ერთდროულად გამოყენება გვჭირდება, ხშირად უმჯობესია `classList`.

6. სრული პრაქტიკული მაგალითი — პროდუქტების ბარათები

შევქმნათ სამი პროდუქტის ბარათი, რომელთა დიზაინს JavaScript-ით შევცვლით.

HTML — `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Product Cards</title>
    <link rel="stylesheet" href="style.css">
    <script src="script.js" defer></script>
</head>
<body>

    <h1 id="page-title">Our Products</h1>

    <div class="card">
        <h2>Laptop</h2>
        <p>Price: $900</p>
    </div>

    <div class="card">
        <h2>Phone</h2>
```

```
        <p>Price: $600</p>
    </div>

    <div class="card">
        <h2>Tablet</h2>
        <p>Price: $350</p>
    </div>

</body>
</html>
```

CSS — `style.css`

```
body {
    font-family: Arial, sans-serif;
    padding: 30px;
    background-color: #f5f5f5;
}

.card {
    padding: 20px;
    margin-bottom: 15px;
    border: 1px solid gray;
    border-radius: 10px;
    background-color: white;
}

.featured {
    background-color: #dbeafe;
    border: 2px solid #2563eb;
}

.expensive {
    color: red;
}

.available {
    color: green;
}
```

JavaScript — `script.js`

```
// 1. მთავარი სათაურის მოძიება
let title = document.querySelector("#page-title");
```

```
// 2. სათაურის სტილის შეცვლა
title.style.color = "darkblue";
title.style.textAlign = "center";
title.style.fontSize = "36px";

// 3. ყველა ბარათის მოძიება
let cards = document.querySelectorAll(".card");

// 4. ბარათების სტილების შეცვლა
cards.forEach((card) => {
    card.style.width = "300px";
    card.style.padding = "25px";
});

// 5. პირველი ბარათის გამორჩევა
cards[0].classList.add("featured");

// 6. მეორე ბარათის ფონის შეცვლა
cards[1].style.backgroundColor = "lightyellow";

// 7. მესამე ბარათისთვის კლასის დამატება
cards[2].classList.add("featured");

// 8. ფასების მოძიება
let prices = document.querySelectorAll(".card p");

// 9. განსხვავებული კლასების მინიჭება
prices[0].classList.add("expensive");

prices[1].classList.add("available");

prices[2].classList.add("available");

// 10. კლასის არსებობის შემოწმება
console.log(cards[0].classList.contains("featured"));
```

მიღებული შედეგი



ამ მაგალითში მოსწავლეები ერთდროულად იყენებენ წინა გაკვეთილებზე ნასწავლ DOM მეთოდებს და ახლად შესწავლილ CSS სტილების მართვის შესაძლებლობებს.

7. პრაქტიკული სავარჯიშოები

სავარჯიშო 1 — სათაურებისა და პარაგრაფების დიზაინის შეცვლა

შექმენით HTML გვერდი, რომელზეც იქნება:

- ერთი `h1` ელემენტი `id="main-title"`;
- სამი `h2` ელემენტი;
- ოთხი პარაგრაფი `class="description"`.

JavaScript-ის გამოყენებით:

1. მოძებნეთ მთავარი სათაური და შეუცვალეთ ტექსტის ფერი ლურჯით.
2. სათაურს მიანიჭეთ `36px` ზომა.
3. სათაური განათავსეთ ცენტრში `textAlign` თვისებით.
4. მოძებნეთ ყველა `h2` ელემენტი და `forEach()`-ის საშუალებით შეუცვალეთ ტექსტის ფერი მწვანით.
5. მოძებნეთ ყველა პარაგრაფი და მიანიჭეთ `20px` ზომა.
6. პირველ და ბოლო პარაგრაფს მიანიჭეთ განსხვავებული ფონის ფერები.

სავარჯიშო 2 — CSS კლასების მართვა

შექმენით HTML გვერდი, სადაც იქნება ოთხი `div` ელემენტი `class="box"`.

CSS-ში შექმენით შემდეგი კლასები:

- `.red-box` — წითელი ფონი და თეთრი ტექსტი.
- `.blue-box` — ლურჯი ფონი და თეთრი ტექსტი.
- `.rounded` — მომრგვალებული კუთხეები.
- `.large` — `250px` სიგანე და `120px` სიმაღლე.

JavaScript-ის გამოყენებით:

1. მოძებნეთ ყველა `box` კლასის მქონე ელემენტი.
2. პირველ ელემენტს დაამატეთ `red-box` კლასი.
3. მეორე ელემენტს დაამატეთ `blue-box` კლასი.
4. მესამე ელემენტს დაამატეთ `rounded` და `large` კლასები.
5. მეოთხე ელემენტს დაამატეთ `red-box` და `rounded` კლასები.
6. `classList.remove()`-ის გამოყენებით პირველ ელემენტს წაუშალეთ `red-box` კლასი.
7. `classList.toggle()`-ით მეორე ელემენტს წაუშალეთ `blue-box` კლასი და შემდეგ ისევ დაუმატეთ.
8. `classList.contains()`-ის გამოყენებით შეამოწმეთ, აქვს თუ არა მესამე ელემენტს `large` კლასი და შედეგი გამოიტანეთ კონსოლში.

სავარჯიშო 3 — სტუდენტების ბარათები

შექმენით სტუდენტების საინფორმაციო გვერდი, სადაც იქნება მინიმუმ 4 ბარათი.

თითოეული ბარათი უნდა შეიცავდეს სტუდენტის სახელს, ასაკსა და სასწავლო პროგრამას.

ყველა ბარათს მიანიჭეთ `class="student-card"`.

JavaScript-ის გამოყენებით:

1. მოძებნეთ ყველა სტუდენტის ბარათი.
2. `forEach()`-ის გამოყენებით ყველა ბარათს მიანიჭეთ `padding`, `margin`, `border` და `borderRadius`.
3. პირველ ბარათს შეუცვალეთ ფონის ფერი.
4. მეორე ბარათს შეუცვალეთ ტექსტის ფერი.
5. CSS-ში შექმენით `.excellent-student` კლასი, რომელიც გამოარჩევს სტუდენტს განსხვავებული ფონითა და საზღვრით.
6. მესამე ბარათს დაამატეთ `excellent-student` კლასი.
7. მეოთხე ბარათს `classList.toggle()` მეთოდით დაამატეთ და შემდეგ წაუშალეთ იგივე კლასი.
8. კონსოლში გამოიტანეთ ყველა სტუდენტის სახელი და შეამოწმეთ, რომელ ბარათს აქვს `excellent-student` კლასი.

სავარჯიშო 4 — პროდუქტის კატალოგი

შექმენით ტექნიკის მაღაზიის მარტივი ვებგვერდი.

გვერდზე უნდა იყოს:

- მთავარი სათაური;
- მინიმუმ 5 პროდუქტის ბარათი;
- თითოეული პროდუქტის სახელი, აღწერა და ფასი;
- საერთო `product-card` კლასი ყველა ბარათისთვის.

JavaScript-ის გამოყენებით:

1. შეცვალეთ მთავარი სათაურის ფერი და ზომა.
2. `querySelectorAll()`-ით მოძებნეთ ყველა პროდუქტის ბარათი.
3. `forEach()`-ით ყველა ბარათს მიანიჭეთ ერთნაირი სტილები.
4. CSS-ში შექმენით `.discount` კლასი, რომელიც პროდუქტის ბარათს განსხვავებული ფონითა და საზღვრით გამოარჩევს.
5. პირველ და მესამე პროდუქტს დაამატეთ `discount` კლასი.
6. მეორე პროდუქტს JavaScript-ით შეუცვალეთ ფონის ფერი.
7. მეოთხე პროდუქტს დაამატეთ და შემდეგ წაუშალეთ `discount` კლასი.
8. მეხუთე პროდუქტზე `classList.contains()` მეთოდით შეამოწმეთ, აქვს თუ არა მას `discount` კლასი.
9. კონსოლში გამოიტანეთ პროდუქტების რაოდენობა და იმ პროდუქტების სახელები, რომლებსაც `discount` კლასი აქვთ.